

Poenget med abstraksjoner

Eller: Hvordan gjøre ting enkle å endre

Agenda

- **Hvorfor abstraksjoner?** ETC – Easy To Change
- **Domenemodellen:** Typer som dokumentasjon
- **Parse, don't validate:** Korrekthet ved grensen
- **Porter:** Kontrakter, ikke implementasjoner
- **Use cases:** Én jobb, ingen avhengigheter nedover
- **Adaptere:** Det som faktisk gjør jobben
- **Gevinsten:** Fra V1 til V2

Hvorfor abstraksjoner?

Ikke fordi det er gøy. Fordi ting endrer seg

"ETC – Easy To Change. That's it. As far as we can tell, every design principle out there is a special case of ETC."
— The Pragmatic Programmer

-  Krav endrer seg (alltid)
-  Vi må kunne teste ting isolert
-  Vi må kunne bytte ut deler uten å rive ned alt

Spørsmålet er ikke "trenger vi abstraksjon?"

Spørsmålet er "gjør denne abstraksjonen koden lettere å endre?"

Domenemodellen: V1 vs V2

V1: Flat og implisitt

- Ingen `Conversation` -modell – bare flate JSON-blobs i databasen
- Feil fanget kun på route-nivå
- `NetworkError(throwable)` som catch-all

V2: Eksplisitt og typet

```
data class Conversation(  
    val id: String,  
    val name: String,  
    val messages: List<MessageV2>,  
    val selectionID: String? = null  
)
```

Samtalen er en førsteklasses borger i domenet.

Parse, don't validate

Håndhev korrekthet **ved grensen** — og stol på typene dine overalt ellers.

```
data class CreateConversationRequest private constructor(
    val query: String,
    val selectionID: String? = null
) {
    companion object {
        operator fun invoke(query: String, selectionID: String? = null) =
            CreateConversationRequest(query = query.sanitized(), selectionID = selectionID)
    }
}

sealed class MessageV2 {
    data class User private constructor(val query: String) : MessageV2() {
        companion object {
            operator fun invoke(query: String) = User(query = query.sanitized())
        }
    }
    // ...
}
```

Privat konstruktør + `operator fun invoke` = **umulig å opprette uten parsing**

Parsing ved HTTP-grensen

```
private data class CreateChatRequestDTO(val query: String, ...) {  
    fun toDomain(): Either<RouteError, CreateConversationRequest> =  
        either {  
            ensure(query.isNotEmpty()) { RouteError.BadRequest("...") }  
            ensure(query.toByteArray().size <= MAX_QUERY_SIZE) { RouteError.BadRequest("...") }  
            CreateConversationRequest(query = query, selectionID = selectionID)  
        }  
}
```

Rå input kommer inn som DTO → valideres og parses til en domenetypen → eller avvises.

Når du har en `CreateConversationRequest`, er den **garantert gyldig**.

Ingen defensive sjekker lenger ned i stacken. Ingen "just in case"-validering.

Sealed classes: Ugyldige tilstander er umulige

```
sealed class MessageV2 {  
    data class User(val query: String) : MessageV2()  
    data class System(  
        val response: XHTML.Node,  
        val relevantDocuments: List<RelevantDocument>  
    ) : MessageV2()  
    data class Error(val message: String) : MessageV2()  
}
```

En melding er **enten** fra bruker, **eller** fra systemet, **eller** en feil. Aldri noe midt imellom.

- ✓ Compileren tvinger deg til å håndtere alle varianter
- ✓ Nye varianter → compile error der du glemmer noe

Typedrevet tilstandsmaskin

```
private fun Conversation.toResponse() =  
    when (this.messages.lastOrNull()) {  
        null          -> ChatResponseDTO.InitialLoading(...)  
        is MessageV2.User    -> ChatResponseDTO.AwaitingReply(...)  
        is MessageV2.System -> ChatResponseDTO.Idle(...)  
        is MessageV2.Error   -> ChatResponseDTO.Failed(...)  
    }
```

Ingen boolske flagg. Ingen `state: String` . Bare typer.

- ⚠️ V1: Frontend måtte gjette tilstand basert på JSON-felter
- ✅ V2: Tilstanden er **uomtviselig** – den følger av siste melding

Ærlig feilhåndtering

```
sealed interface AISearchErrorV2 {  
    data class Validation(val message: String) : AISearchErrorV2  
    data class NotFound(val message: String) : AISearchErrorV2  
    data object RateLimited : AISearchErrorV2  
    data class ServiceUnavailable(...) : AISearchErrorV2  
    data class Timeout(...) : AISearchErrorV2  
    data object TooLargeHeader : AISearchErrorV2  
    data class Internal(...) : AISearchErrorV2  
}
```

V1: `NetworkError(throwable)` – hva gikk egentlig galt?

V2: Hver feiltype er eksplisitt – koden *forteller* deg hva som kan skje

 `Either<AISearchErrorV2, T>` — funksjoner som ikke lyver

Porter

En port er en **kontrakt** — et interface som definerer
hva systemet trenger, ikke *hvordan* det løses.

LLMPort

```
interface LLMPort {  
    suspend fun chat(  
        userID: Int,  
        conversation: Conversation  
    ): Either<AISearchErrorV2, Conversation>  
}
```

Det er hele greia.

- Ingen HTTP-klienter, ingen JSON-parsing, ingen retry-logikk
- Bare: «gi meg en samtale tilbake, eller en feil»

CachePort

```
interface CachePort {  
    suspend fun createConversation(  
        userID: Int, query: String, selectionID: String?  
    ): Either<AISearchErrorV2, Conversation>  
  
    suspend fun getConversation(  
        userID: Int, conversationID: String  
    ): Either<AISearchErrorV2, Conversation>  
  
    suspend fun updateConversation(  
        userID: Int, conversation: Conversation  
    ): Either<AISearchErrorV2, Unit>  
  
    suspend fun deleteConversation(  
        userID: Int, conversationID: String  
    ): Either<AISearchErrorV2, Unit>  
    // + list-operasjoner  
}
```

📦 CRUD-kontrakt for samtaler — implementasjonen kan være hva som helst

Use cases

En use case gjør **én ting**.

Den kjenner kun porter — aldri konkrete implementasjoner.

PostMessage

```
class PostMessage(  
    private val llmPort: LLMPort,  
    private val cachePort: CachePort,  
    private val serverEventEmitter: ServerEventEmitter,  
    private val coroutinePool: CoroutinePool  
) {  
    suspend operator fun invoke(  
        userID: Int, sessionId: String,  
        conversationID: String, query: String  
    ): Either<AISearchErrorV2, Conversation> =  
        either {  
            val conversation =  
                cachePort.getConversation(userID, conversationID).bind()  
                    .dropErrors()  
                    .withMessage(MessageV2.User(query))  
                    .also { cachePort.updateConversation(userID, it).bind() }  
  
            coroutinePool.launch { /* LLM-kall i bakgrunnen */ }  
            conversation  
        }  
}
```

Hva skjer i PostMessage?

1. 📥 Hent samtalen fra cache (CachePort)
2. 💬 Legg til brukerens melding
3. 💾 Lagre oppdatert samtale (CachePort)
4. 🚀 Start LLM-kall i bakgrunnen (LLMPort + CoroutinePool)
5. 🔄 Returner samtalen umiddelbart

Use casen **orkestrerer** — den vet ikke om cache er Postgres eller en `HashMap`, og heller ikke om LLM-et er Lovdata API eller en mock.

Kobling

Rutene kobler alt sammen — **eneste stedet**
som vet hvilke adaptere som brukes.

Dependency injection i rutene

```
class AISearchRoutesV2 @Inject constructor(  
    application: Application,  
    llmPort: LLMPort,  
    cachePort: CachePort,  
    serverEventEmitter: ServerEventEmitter  
) {  
    private val coroutinePool = CoroutinePool(maxConcurrency = 50)  
  
    private val createConversation = CreateConversation(llmPort, cachePort, ...)  
    private val getConversation    = GetConversation(cachePort)  
    private val postMessage       = PostMessage(llmPort, cachePort, ...)  
    private val retryLastMessage  = RetryLastMessage(llmPort, cachePort, ...)  
    private val deleteConversation = DeleteConversation(cachePort)  
}
```

✿ Hver use case får portene den trenger — ferdig.



Ekte adapter: LovdataApiAdapter

```
class LovdataApiAdapter @Inject constructor(  
    private val httpClient: HttpClient,  
    private val documentService: DocumentService  
) : LLMPort {  
  
    override suspend fun chat(userID: Int, conversation: Conversation) =  
        either {  
            val docs = findRelevantDocuments(userID, ...).bind()  
            val request = buildChatRequest(userID, conversation, docs)  
            val (response, name) = generateResponseAndName(request).bind()  
            conversation  
                .withNameIfNotNull(name)  
                .withMessage(MessageV2.System(response, docs))  
        }  
    // 300+ linjer: HTTP-kall, DTO-mapping, feilhåndtering...  
}
```

🏢 All kompleksiteten er **innkapslet** bak `LLMPort`

Mock-adapter: **MockLLMAdapter**

```
class MockLLMAdapter : LLMPort {  
    override suspend fun chat(  
        userID: Int, conversation: Conversation  
    ): Either<AISearchErrorV2, Conversation> =  
        conversation.withMessage(mockResponse()).right()  
}
```

✅ Samme interface — **5 linjer** i stedet for 300+

Use casene merker **ingen forskjell**. Det er hele poenget.

Hva fikk vi?

	V1	V2
Struktur	Én stor <code>AISeachService</code>	Ports + Adapters + Usecases
Lagring	Flat JSON-blob	Normaliserte tabeller
Feilhåndtering	Kun på route-nivå	I domenemodellen (<code>Either</code>)
Samtaler	Finnes ikke	Førsteklasses aggregat
Bytte LLM/søkeimplementasjon	Skrive om servicen	Skrive én ny adapter
Testing	Mock hele servicen	Injiser mock-adapter

Når kravene endrer seg...

"Vi skal bytte fra lovdata-api til direktesøk eller et helt annet oppsett"

→ Skriv en ny adapter som implementerer `LLMPort` . Ferdig.

"Vi trenger en annen lagringsløsning"

→ Skriv en ny adapter som implementerer `CachePort` . Ferdig.

"Vi vil legge til en ny funksjon"

→ Lag en ny use case. Eksisterende kode forblir urørt.

Ingen endring i domenelogikk. Ingen endring i andre adaptere.

Testing uten smerte 🩺

Begge implementerer det samme grensesnittet:

```
interface LLMPort {  
    suspend fun chat(  
        userID: Int,  
        conversation: Conversation  
    ): Either<AISearchErrorV2, Conversation>  
}
```

4 linjer som definerer kontrakten.

Ingen `every { }`, `verify { }`, `@MockBean`.

Bare en klasse som implementerer et interface.

ETC som daglig designvane 🧭

"ETC is a value, not a rule. [...] It's the ultimate arbiter between design alternatives."

— The Pragmatic Programmer

Ikke et arkitekturmønster du "følger" — en daglig vane:

- **Før du skriver kode:** *"Gjør dette det lettere å endre senere?"*
- **Når du er i tvil:** *"Hvilken løsning er enklere å endre?"*
- **Når noen spør hvorfor:** *"Fordi endring er billig."*

Arkitektur handler ikke om å være "ren" eller "riktig".

Det handler om å gjøre **neste endring billig**.

Takk! 🙏

Spørsmål?